SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 –Dataset Intégration Task

Course Code:	DSA0502	**Course Title:**	Query Processing for Data Science
CO Assessed:	CO3	**Assessment:**	Tool 3 - Dataset Intégration Task
Weightage:		**Total Marks:**	

CO3 Statement: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex. (BL5 and BL6).

Student Name:	Sandra Sudevan	**Reg. No.:**	192424422
Section / Batch:	2024	**Date of Submission:**	16l08l26
Faculty In-charge:	**Dr. B.Shyamala Gowri**	**Project Mode:**	Individual Submission

Code of Conduct

I Sandra Sudevan(192424422) (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Sandrasudevan

Dataset Intégration Task

Question 4 – Student Dataset Integration

A college maintains student information in **Admission, Examination, and Attendance datasets**. Student names have spelling variations, register numbers have different formats, dates are inconsistent, email addresses may be invalid, and duplicate student records exist.

Task: Using Python/Pandas, clean and integrate the datasets by applying **data**

formatting, missing-value handling, RegEx matching, fuzzy matching, duplicate identification, normalization, and standardization. Create a master student dataset and write a reusable Python cleanup script. Test the script with a new set of student records.

Aim

To clean and integrate Admission, Examination, and Attendance student datasets using **Python/Pandas**, applying data formatting, missing-value handling, RegEx validation, fuzzy matching, duplicate identification, normalization, and standardization, and to create a reusable master student dataset.

Techniques Used

- **Normalization:** Student names and register numbers
- **Standardization:** Names, emails, dates and register numbers
- **RegEx:** Email validation and register-number formatting
- **Missing-value handling:** Invalid/missing values converted to `NaN`
- **Fuzzy matching:** `SequenceMatcher` for spelling variations
- **Duplicate identification:** Exact duplicate detection
- **Integration:** Admission + Examination + Attendance
- **Aggregation:** Average examination marks and attendance
- **Testing:** New student records are processed using the same functions

Expected input

Dataset	Important columns
Admission.csv	RegisterNo, Name, DOB, Email
Examination.csv	RegisterNo, Name, ExamDate, Marks
Attendance.csv	RegisterNo, Name, AttendanceDate, Attendance

CODE:

```
import pandas as pd
import re
from difflib import get_close_matches

a = pd.read_csv("Admission.csv")
e = pd.read_csv("Examination.csv")
t = pd.read_csv("Attendance.csv")

# Clean data
for df in [a, e, t]:
    df["Name"] = df["Name"].str.strip().str.lower().str.title()
    df["RegisterNo"] = df["RegisterNo"].astype(str).str.upper()
```

```
    df.drop_duplicates(inplace=True)

# Normalize register number
def reg(x):
    n = re.findall(r'\d+', x)
    return "REG" + n[0] if n else None

for df in [a, e, t]:
    df["RegisterNo"] = df["RegisterNo"].apply(reg)

# Email validation
a["Email"] = a["Email"].str.lower()
a.loc[~a["Email"].str.match(
    r'^[\w.-]+@[\w.-]+\.\w+$', na=False
), "Email"] = pd.NA

# Date formatting
for df in [a, e, t]:
    for col in ["DOB", "ExamDate", "AttendanceDate"]:
        if col in df:
            df[col] = pd.to_datetime(df[col], errors="coerce")

# Fuzzy matching
for i in e.index:
    match = get_close_matches(
        e.loc[i, "Name"],
        a["Name"].tolist(),
        n=1,
        cutoff=0.8
    )
    if match:
        e.loc[i, "Name"] = match[0]

# Merge
master = a.merge(
    e, on=["RegisterNo", "Name"], how="outer"
)

master = master.merge(
    t, on=["RegisterNo", "Name"], how="outer"
)

# Missing values
```

```
master = master.astype(object).fillna("Not Available")


# Save
master.to_csv("Master_Student.csv", index=False)


print("Master Student Dataset Created!")
print(master)
```

OUTPUT:

```
PS C:\Users\sandr\OneDrive\Desktop\StudentProject> python student.py
Master Student Dataset Created!
  RegisterNo          Name                  DOB            Email      AttendanceDate     Attendance            ExamDate          Marks
0     REG101   Anita Sharma  2005-12-03 00:00:00  anita@gmail.com  2026-01-08 00:00:00           92.0       Not Available  Not Available
1     REG101  Anitha Sharma        Not Available    Not Available        Not Available  Not Available  2026-10-07 00:00:00           88.0
2     REG102    Rahul Kumar        Not Available  rahul@gmail.com        Not Available           95.0       Not Available           91.0
3     REG103     Priya Nair  2006-05-01 00:00:00    Not Available        Not Available           89.0       Not Available           84.0
4     REG104     Kavin Raaj        Not Available    Not Available        Not Available  Not Available  2026-10-07 00:00:00           79.0
5     REG104      Kavin Raj        Not Available  kavin@gmail.com  2026-01-08 00:00:00           87.0       Not Available  Not Available
PS C:\Users\sandr\OneDrive\Desktop\StudentProject>
```

Question 5 – Healthcare Patient Dataset Integration

A hospital stores patient information in **Registration, Laboratory, and Billing datasets**. Patient names may contain spelling errors, phone numbers use different formats, emails may be invalid, dates have different formats, and duplicate patient records exist across departments.

Task: Develop a Python program to clean and integrate the three datasets. Identify bad data and missing values, validate fields using **RegEx, detect duplicates, perform fuzzy matching, standardize dates and categorical values, identify numerical outliers, and merge the datasets**. Save the final cleaned dataset and create a script that can be tested on newly received patient data.

Aim

To clean and integrate Registration, Laboratory, and Billing datasets using Python/Pandas by identifying bad data, missing values, duplicates, invalid fields, spelling errors, outliers, and standardizing the data.

Techniques Used

1. Data cleaning and formatting
2. Missing-value handling
3. RegEx validation
4. Duplicate detection and removal
5. Fuzzy matching
6. Phone-number normalization
7. Date standardization
8. Categorical-value standardization
9. Numerical outlier detection using IQR
10. Dataset merging/integration
11. Master dataset creation

CODE:

```
import pandas as pd
```

```
import re
from difflib import get_close_matches

# Read datasets
r = pd.read_csv("Registration.csv")
l = pd.read_csv("Laboratory.csv")
b = pd.read_csv("Billing.csv")

# Standardize column names
for df in [r, l, b]:
    df.columns = df.columns.str.strip()

# Clean names and phone numbers
for df in [r, l, b]:
    df["Name"] = df["Name"].str.strip().str.lower().str.title()
    df.drop_duplicates(inplace=True)

r["Phone"] = r["Phone"].astype(str).str.replace(
    r"\D", "", regex=True
)

# Email validation
r["Email"] = r["Email"].str.lower()
r.loc[~r["Email"].str.match(
    r"^[\w.-]+@[\w.-]+\.\w+$", na=False
), "Email"] = pd.NA

# Date standardization
for df in [r, l, b]:
    for c in ["DOB", "TestDate", "BillDate"]:
        if c in df:
            df[c] = pd.to_datetime(df[c], errors="coerce")

# Categorical standardization
if "Gender" in r:
    r["Gender"] = r["Gender"].str.strip().str.upper()

# Fuzzy matching names
for i in l.index:
    m = get_close_matches(
        l.loc[i, "Name"],
        r["Name"].tolist(),
        n=1, cutoff=0.8
```

```
    )
    if m:
        l.loc[i, "Name"] = m[0]

# Detect numerical outliers
if "Amount" in b:
    q1 = b["Amount"].quantile(.25)
    q3 = b["Amount"].quantile(.75)
    iqr = q3 - q1
    b["Outlier"] = (
        (b["Amount"] < q1 - 1.5 * iqr) |
        (b["Amount"] > q3 + 1.5 * iqr)
    )

# Merge datasets
master = r.merge(
    l, on=["PatientID", "Name"], how="outer"
)
master = master.merge(
    b, on=["PatientID", "Name"], how="outer"
)

# Missing values
master = master.astype(object).fillna("Not Available")

# Save
master.to_csv("Master_Patient.csv", index=False)

print("Master Patient Dataset Created!")
print(master)
```

OUTPUT:

```
PS C:\Users\sandr\OneDrive\Desktop\HealthcareProject> python patient.py
Master Patient Dataset Created!
  PatientID         Name                  DOB Gender  ... Result            BillDate Amount Outlier
0      P101  Anita Sharma  1985-12-03 00:00:00      F  ...    120  2026-01-08 00:00:00    500   False
1      P102   Rahul Kumar        Not Available      M  ...     95        Not Available    750   False
2      P103    Priya Nair  1978-05-01 00:00:00      F  ...    210        Not Available  50000   False

[3 rows x 12 columns]
```